

{ }

< / >

Java
Today
A Better
Tomorrow



Code
Ideas
Progress

Learn
Practice
Build



Programming Language



Language

- Any medium to communicate is called a language.



Programming Language

- Any language that is used to communicate with a computer or a machine is called Programming Language. Programming language is the form of set of instructions.



Types of Programming Language

1

Low Level (Binary Language [0,1]).

- A computer can directly understand assembly language.

2

Assembly Language (Mod, Bit and other)

- It is in the form of mnemonics (easily understood words).
- It is directly converted into machine language by an assembler.



James Gosling and Patrick Naughton

Employees of Sun Microsystems, with support from **Bill Joy** — co-founders of Sun Microsystems.



Low Level Language

- Low level language is the first programming language which was used to instruct computer.
- It is in binary form [0,1], so it is called **Binary Language**.
- A computer can directly understand low level language, but it is difficult for a human to understand.

e.g.:

0	→	off
1	→	on



Assembly Language vs. Machine/Low Level Language

Assembly Language

- A computer can directly understand assembly language.
- It is in the form of mnemonics (easily understood words).
- It is directly converted into machine language by an assembler.

Machine/Low Level Language

- A computer can directly understand machine language.
- It is in the form of 0s and 1s (binary).
- It is not easy for a human to understand.
- It is directly executed by the computer.



Key Point:

A computer cannot understand assembly language directly. So, assembly language is used as a translator which takes assembly language and converts it into low level language (machine language).



High Level Language

- A language that a human can understand directly is called high level language.
- It is a high level language because it is easy to understand.
- It makes programming very easy to understand.

e.g.: 1972 → C language

A computer cannot understand high level language code directly, so compiler and interpreter are used. In C, a translator which takes high level code and converts into low level code is used.



Every high level language will have own compiler.

* Java :

- Java is a high level, object oriented language [platform independent] computer programming language, which features object oriented concepts [Encapsulation, Inheritance, Polymorphism, Abstraction].

* Java was developed by James Gosling in 1991 under Sun Microsystems and it was released in 1995.

1995 ⇒ Java beta version.

1996 ⇒ Java 1.0.0 (JDK 1.0.0).

1997 ⇒ Java 2.0.0 (JDK 2.0.0).

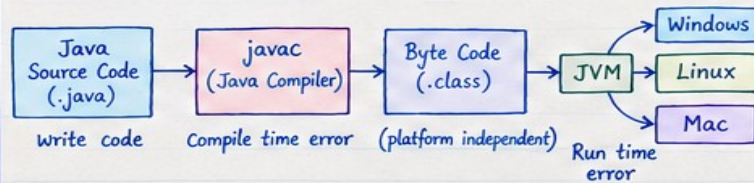
⋮

March 2022 ⇒ Java 18.0.0 (JDK 18.0.0).

→ 20

* In 2010 ORACLE bought Sun Microsystems.

* Java is a platform independent programming language.

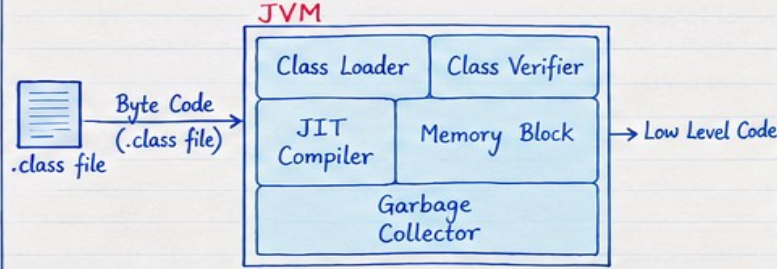


* When an application is developed by using Java language then the .java file is saved as first step and it is called Java Compilation, which takes .java file and if it is correct then next level code will be generated by compiler which is called Byte Code or .class file. Byte Code is also called Magical or Intermediate Code, which a computer or JVM will understand directly.

* JVM : JVM stands for Java Virtual Machine. It works as a translator which takes byte code and converts into the local code, which a computer can understand. In this manner Java becomes a platform independent. So it is called "WORA" [Write Once RUN Anywhere] language.

* JRE, JDK, JIT Compiler :

1) Java Virtual Machine :



⇒ JVM stands for Java Virtual Machine. JVM is called Virtual because JVM doesn't exist alone. JVM needs a body for implementation.

2) The Components of JVM are :

(a) Class Loader : Loads the class file inside JVM.

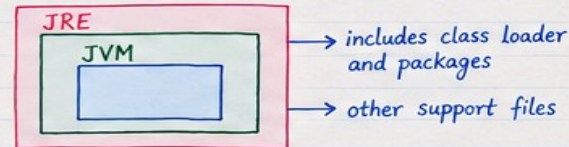
(b) Class Verifier : Will reads loaded class file line by line and checks that generated byte codes are correct. So that overall efficiency can be improved.

(c) JIT Compiler : Used to convert byte code to native code to improve the performance.

(d) Memory Block : Takes memory area for all memory related tasks.

(e) Garbage Collector : Deletes unwanted memory so that execution can be maintain.

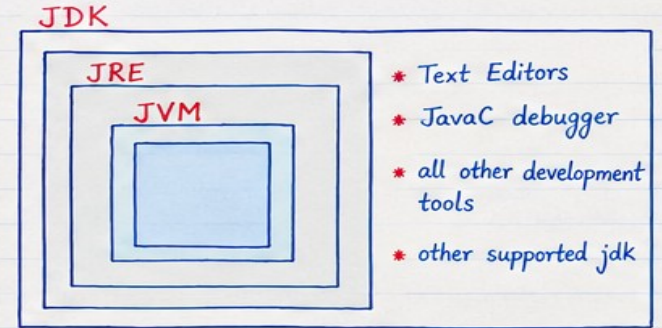
* JRE :



JRE stands for Java Runtime Environment. JRE provides implementation to JVM. So JVM is physically present inside JRE. JRE containing JVM along with class files and packages and library files. By using JRE we can only run the applications but we can't do development parts.

* JDK (Java Development Kit) :

1) Java Virtual Machine :



JDK stands for Java Development Kit. It combines JRE along with development tools which are needed by a programmer to develop an application. By using JDK we can run the application as well as we can do development part.

JDK is platform dependent So Different platform have different JDK.

* JIT (Just In Time) Compiler :

JIT is a part of JVM. It is used inside JVM to improve its efficiency.

* JRE, JDK, JIT Compiler :

① JRE - provides runtime environment to JVM.

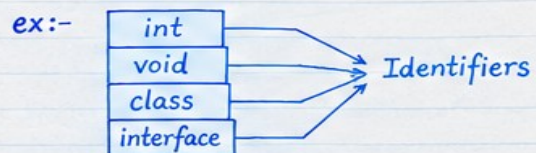
② JDK - provides JRE along with development tools and other support files.

③ JIT - used inside JVM to improve efficiency.

* Identifiers :

In Java, identifiers are the names of classes, variables, methods, constructors, packages, interfaces or any member in the program. Identifiers are used to give a name to a program element.

→ Keywords are special types of identifiers but they are not used as identifiers. Identifiers are used only for identification purpose.



Note: Every identifier must be a valid name (not a keyword).

* Rules for Identifiers :

- ① For identifiers, we can use alphabets (a-z or A-Z), numbers (0-9) and special characters (_ and \$) but not start with a number.
- ② Identifiers must not start with a number.
- ③ No space allowed for identifiers.
- ④ Keywords cannot be used as an identifier.

eg:-

1) FirstProgram123	✓
2) FirstProgram@123	✗
3) FirstProgram\$123	✓
4) 123FirstProgram	✗
5) _123FirstProgram	✓
6) \$FirstProgram	✓
7) First Program	✗
8) public	✗
9) Public	✗
10) Public1	✓

→ java is a case sensitive language so uppercase and lowercase are different.

* Conventions for Identifiers :

Conventions are set of guidelines for identifier for good programming practice.

* Conventions for a Class Name :

- Class name should be written in 'UpperCamelCase'.
- It should start with a capital letter.
- Example: StudentRecord, Employee, FirstProgram

* Conventions for a Method Name :

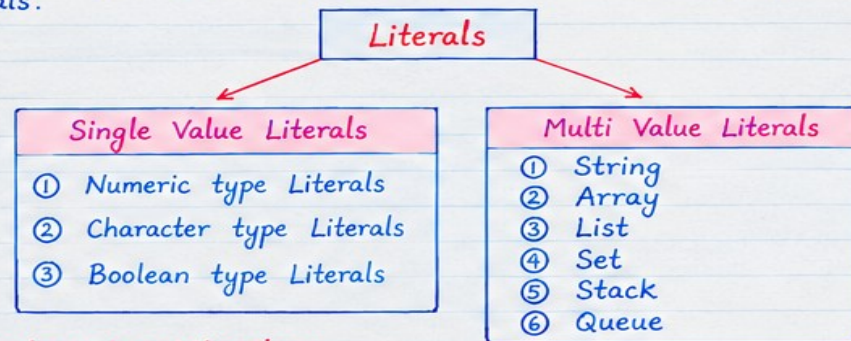
- Method name should be written in 'lowerCamelCase'.
- It should start with a small letter.
- Example: displayInfo(), calculateSum(), getData()

* Conventions for a Variable Name :

- Variable name should be written in 'lowercase'.
- It should start with a small letter.
- Example: age, name, price, total

* Literals :

The value of different type of data is stored in a program is called Literals.



① Single Value Literals :

- Numeric type Literals :
Represents numeric value (integer, floating point, etc).
Example: 10, 20, 3.14, 0, -5
- Character type Literals :
Represents a single character and written in single quotes.
Example: 'a', 'z', '1', '\$'
- Boolean type Literals :
Represents boolean value (true or false).
Example: true, false

* 1. Data :

Data is any piece of information. In Java, data are the names of constants, variables, and other values used in a program.

* 2. Data Types :

In Java, data types are the names of classes, variables, methods, constructors, packages, interfaces or any member in the program. Identifiers are used to give a name to a program element.

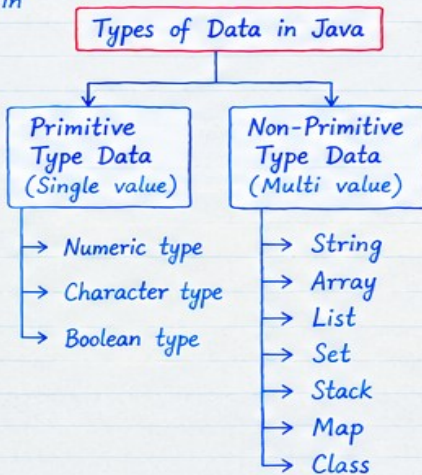
→ Keywords are special types of identifiers but they are not used as identifiers. Identifiers are used only for identification purpose.

* 3. Types of Data :

In Java, data are divided into two types :

(a) Primitive type data (Single value data)

(b) Non-primitive type data (Multi-value data)



Design by Sarafraj Navaz

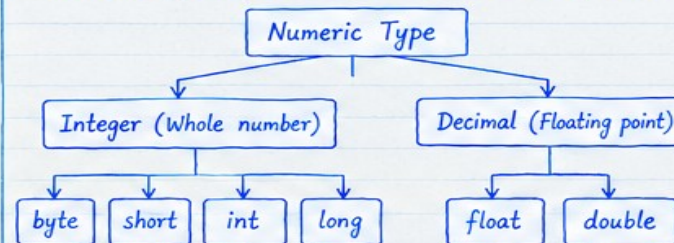
* 4. Primitive Type Data :

Primitive types are the basic building blocks of data. They are single value data types and are divided into:

1) Numeric type 2) Character type 3) Boolean type

(a) Numeric Type :

Numeric type is further divided into two categories :



Design by Sarafraj Navaz

(b) Character Type :

Character type is used to store single character. It is represented by 16 bit.

Example : `char ch = 'A';`

(c) Boolean Type :

Boolean type is used to store true or false value.

Example : `boolean flag = true;`

* 5. Non-Primitive Type Data :

Non-primitive types store multiple values or complex data structures.

Example : String, Array, List, Set, Stack, Map, Class.

* 6. Data Type Size and Range :

Data Type	Size (bits)	Size (bytes)	Range
boolean	1	1 bit	0 to 1
byte	8	1 byte	-128 to 127
short	16	2 bytes	-32,768 to 32,767
int	32	4 bytes	-2,147,483,648 to 2,147,483,647
long	64	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	32	4 bytes	-3.402E38 to 3.402E38 (6-7 digits)
double	64	8 bytes	-1.797E308 to 1.797E308 (15-16 digits)
char	16	2 bytes	0 to 65,535 (Unicode)

* 7. Character Representation :

ASCII :

- Used for English characters.
- Range : 0 - 127.
- Example : 'A' = 65

Unicode :

- Used for all languages and symbols.
- Range : 0 - 65,535 (16 bit).
- Example : 'A' = 65, 'अ' (Hindi) = 2309

Design by Sarafraj Navaz

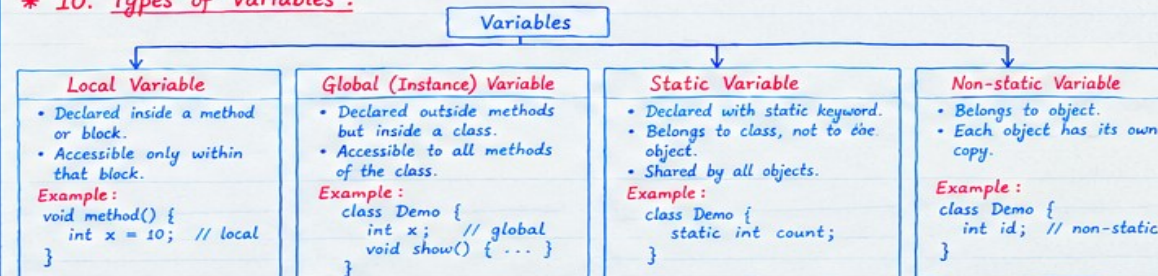
* 8. Variables :

A variable is a container that holds a value. The value can be changed during program execution.

* 9. Declaration, Initialization and Declaration + Initialization :

- 1) Declaration : `int age;`
- 2) Initialization : `age = 25;`
- 3) Declaration + Initialization : `int age = 25;`

* 10. Types of Variables :



JAVA UNIT 1 — PROGRAM COMMENTS

1. Overview

Comments should be used to give overviews of code and provide additional information that is not readily available in the code itself. Comments should contain only information that is relevant to reading and understanding the program.

Types of Comment

1. Implementation Comments

- Block Comments
- Single Line Comment
- Trailing Comments
- End-of-Line Comments

2. Documentation Comments

1. Implementation Comments:

Implementation comments are meant for commenting out code or for comments about the particular implementation. Programs can have four styles of implementation comments.

1 Block Comments

- Used to provide descriptions of files, methods, data structures and algorithms.
- A block comment should be preceded by a blank line to set it apart from the rest of the code.

Syntax: `/* ... */`

Example:

```
/*  
 * Here is a block comment.  
 */
```

2 Single Line Comment

- A single-line comment should be preceded by a blank line.
- Handle the condition.

Syntax: `// ...`

Example:

```
// Handle the condition.
```

3 Trailing Comments

- Very short comments can appear on the same line as the code they describe, but should be shifted far enough to separate them from the statements.

Example:

```
return TRUE; /* special case */
```

4 End-of-Line Comments

- `//` can comment out a complete line or only a partial line.
- It shouldn't be used on consecutive multiple lines for text comments.

Example:

```
if (f > 1) {  
    // Do a double-flip.  
}  
else {  
    return false; // Explain why here.  
}
```

2. Documentation Comments (Doc Comments):

- Java-only comments delimited by `/** ... */`.
- Can be extracted to HTML files using the javadoc tool.
- Used to describe the specification of the code, from classes, interfaces, constructors, methods and fields.
- Each doc comment is set inside the comment delimiters `/** ... */`, with one comment per class, interface, or member.
- Should be placed immediately before the declaration.

Syntax: `/** ... */`

Example:

```
/**  
 * The Example class provides...  
 */
```



Quick Revision:

`/* ... */` = Block comment
`//` = Single-line / End-of-line comment
`/** ... */` = Documentation / Javadoc comment

1. Java Array - Basics

1.1 What is an Array?

- An array is a container object that holds a fixed number of values of a single type.
- It is a collection of elements of the same data type stored in contiguous memory locations.
- Elements are accessed using an index.

1.2 Key Features

- Homogeneous : all elements are of the same type.
- Fixed size : size is decided at creation time.
- 0-based indexing : first element has index 0.
- length property : gives the number of elements.

★ Important

- Array size is fixed after creation.
- Index starts from 0.
- Length gives the number of elements.

1.3 Declaration and Creation

Syntax:

```
dataType[] arrayName; // declaration
arrayName = new dataType[size]; // creation
```

Example:

```
int[] nums;
nums = new int[5];
```

1.4 Initialization (at the time of creation)

Syntax:

```
dataType[] arrayName = {value1, value2, ...};
```

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
```

Memory Representation

- Array elements are stored in contiguous memory locations.
- Each element can be accessed using its index.

2. Types of Arrays

2.1 Single-Dimensional Array

- A single-line comment should be preceded by `//`.
- A trailing comment can appear at the end of the line after the code.
- End-of-line comments can be used to comment out a complete line or part of a line.

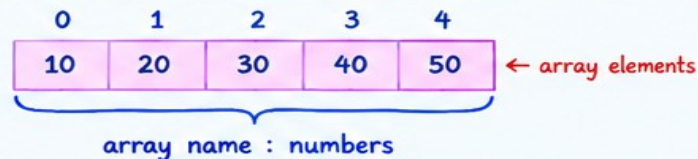
Syntax:

```
dataType[] arrayName = new dataType[size];
```

Example:

```
int[] numbers = new int[5]; // create array
numbers[0] = 10; // assign values
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

Diagram: Single-Dimensional Array (size = 5)



★ Important

- Index starts from 0.
- Array size is fixed after creation.
- You can access elements using `[index]`.

3. Two-Dimensional Array

3.1 Definition

- A two-dimensional array is a collection of elements arranged in rows and columns.
- It can be thought of as a table (matrix).

3.2 Declaration and Initialization

Syntax (declaration):

```
dataType[][] arrayName = new dataType[row][column];
```

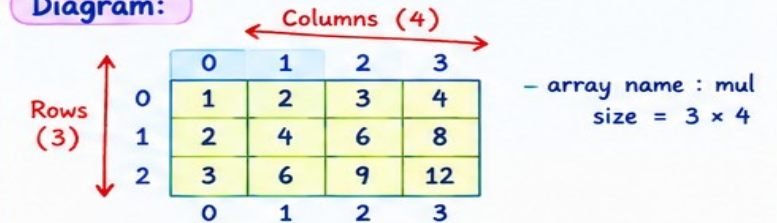
Syntax (with initialization):

```
dataType[][] arrayName = { {v11, v12, ...},
                             {v21, v22, ...},
                             ... };
```

Example - Multiplication Table (3 × 4)

```
int[][] mul = { {1, 2, 3, 4},
                 {2, 4, 6, 8},
                 {3, 6, 9, 12} };
```

Diagram:



3.3 Multidimensional Arrays

- Arrays with more than two dimensions are called multidimensional arrays (e.g., 3-D, 4-D, ...).
- Declared using multiple pairs of square brackets.

Example (3-D array):

```
int[][][] arr = new int[2][3][4];
// 2 blocks, each 3 rows and 4 columns
```


1. Java – Unit 1 : Operators (Page 1)

1.1 Introduction

- Operators are special symbols used to instruct the Java compiler to perform an **operation** on one, two, or three **operands**, and then return a **result**.
- Operators take one or more arguments, or **operands**, and produce a value.
- Java includes a set of 44 different operators. Most programs will need only part of the 44 distinct operators.

1.2 Operator

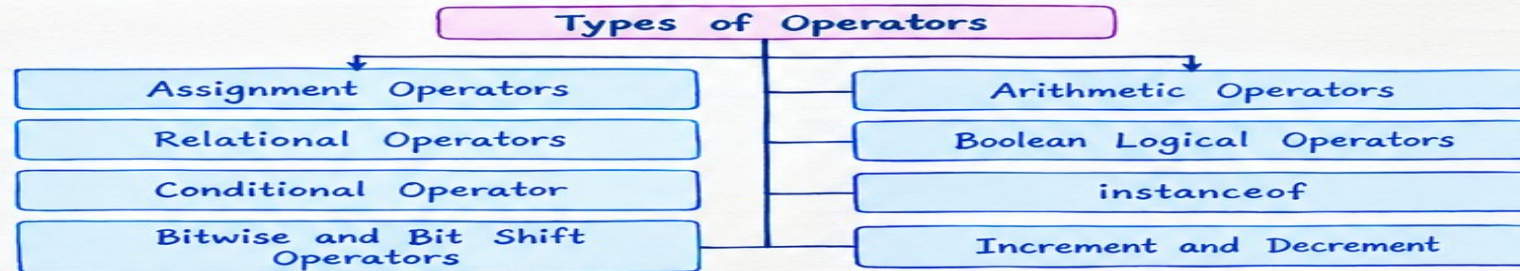
An operator is a special symbol used to perform a specific task on operands.

Example:

$a + b \rightarrow$ addition operator

1.3 Types of Operators

Java provides a rich set of operators to manipulate variables. One can divide all the Java operators into the following groups.



1.4 1) Assignment Operators

Assignment operators set the value of a variable to a literal value or to the value of another variable. Once a variable has been declared, we can assign a value to it using the assignment operator. The value on the right side of the assignment operator is evaluated and then the result is assigned to the variable on the left.

Syntax:

```
var = value; // assign value
```

For example,

```
int var1 = 10;
var1 = var1 + 10; // right side evaluated first
var1 now equals 20.
```

1.5 2) Multiple Assignment Operators

Multiple assignment operators can be used within a single statement.

Syntax:

```
var1 = var2 = var3 = 50;
```

The assignment operator has "right associativity", which means that the right-most assignment operator is performed first.

1.6 3) Arithmetic Operators

Arithmetic operators perform mathematical computations on operands. They include: addition (+), subtraction (-), multiplication (*), division (/) and modulus (%).

Examples:

```
int a = 10, b = 5;
System.out.println(a + b); // 15
System.out.println(a - b); // 5
System.out.println(a * b); // 50
System.out.println(a / b); // 2
System.out.println(a % b); // 0
```

★ **+** (Addition / String Concatenation)

⚡ If one of the operands is a String, the + operator performs string concatenation.

Example:

"Java" + "Programming" → "JavaProgramming"

★ **%** (Modulus / Remainder)

⚡ It returns the remainder after division.

Example:

10 % 3 → 1

1. User Input in Java

Page 1

1.1 3 Steps to Take Input

1. Import the Scanner class from java.util package.
2. Create a Scanner object.
3. Use the object's methods to take input from the user.

1.2 Import Scanner

```
import java.util.Scanner; // import Scanner class
```

1.3 Scanner Object

```
Scanner sc = new Scanner(System.in); // create Scanner object
// System.in is the input stream
```

1.4 Input Methods

Scanner class provides different methods to take input of various data types.

Data Type	Method
byte	nextByte()
short	nextShort()
int	nextInt()
long	nextLong()
float	nextFloat()
double	nextDouble()
String	next() (reads one word) nextLine() (reads whole line)
char	next().charAt(0) (reads single character)
boolean	nextBoolean()

* Note:

- next() reads only one word.
- nextLine() reads the whole line.
- next() returns String, so use Integer.parseInt() etc. for numbers.

1.5 Scanner Package Diagram



Example: `Scanner sc = new Scanner(System.in);` // ready to take input

2. Conditional Operator

Page 2

2.1 Definition

- The conditional operator is a ternary operator used to evaluate a condition and return one of two values.
- It is a shorthand form of the if....else statement.

2.2 Syntax

```
condition ? expression1 : expression2
```

if condition is true

if condition is false

2.3 Example

```
int a = 10, b = 20;
int var = (a > b) ? a : b; // var = 20
```

2.4 Odd or Even Number

```
int num = 7;
String result = (num % 2 == 0) ? "Even" : "Odd";
System.out.println(result); // Odd
```

Output:
Odd

2.5 Divisible by 24

```
int num = 48;
String res = (num % 24 == 0) ? "Divisible by 24" : "Not Divisible";
System.out.println(res); // Divisible by 24
```

Output:
Divisible
by 24

2.6 Last Digit 6

```
int num = 456;
String res = (num % 10 == 6) ? "Ends with 6" : "Does not end with 6";
System.out.println(res); // Ends with 6
```

Output:
Ends with 6

2.7 Lowercase Alphabet

```
char ch = 'b';
String res = (ch >= 'a' && ch <= 'z') ? "Lowercase" : "Not Lowercase";
System.out.println(res); // Lowercase
```

Output:
Lowercase

3. Logical Operators

Page 3

3.1 Definition

- Logical operators are used to perform logical operations on boolean values (true/false).
- Java provides three logical operators: AND (&&), OR (||) and NOT (!).

3.2 Types of Logical Operators

Operator	Symbol	Description	Example
AND	&&	Both conditions must be true.	<code>a && b</code>
OR		At least one condition must be true.	<code>a b</code>
NOT	!	Reverses the value.	<code>!a</code>

3.3 Truth Tables

AND (&&)		
A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

OR ()		
A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

NOT (!)	
A	!A
true	false
false	true

3.4 Short-Circuit Behavior

- In &&, if the first operand is false, the second operand is not evaluated.
- In ||, if the first operand is true, the second operand is not evaluated.

3.5 Examples

1) AND (&&)
`int a = 5, b = 10;`
`boolean result = (a > 0 && b > 0);`
`System.out.println(result);`

Output:
true

2) OR (||)
`int a = 5, b = 10;`
`boolean result = (a < 0 || b > 0);`
`System.out.println(result);`

Output:
true

3) NOT (!)
`boolean flag = false;`
`System.out.println(!flag);`

Output:
true

4. Increment / Decrement and Related Operators

Page 4

4.1 Increment and Decrement Operators

These operators are used to increase or decrease the value of a variable by 1. They can be used in prefix or postfix form.

Operator Type	Name	Example
++	Pre-increment	<code>++a</code>
--	Pre-decrement	<code>--a</code>
++	Post-increment	<code>a++</code>
--	Post-decrement	<code>a--</code>

4.5 Example - Pre vs Post Increment

a) Pre-increment (++a)

```
int a = 10;
int b = ++a;
System.out.println(a); // 11
System.out.println(b); // 11
```

Output:
11
11

b) Post-increment (a++)

```
int a = 10;
int b = a++;
System.out.println(a); // 11
System.out.println(b); // 10
```

Output:
11
10

4.3 Assignment Operator (=)

Assigns the value on the right side to the variable on the left side.

Example: `int a = 10;`
`a = 20;`
`System.out.println(a);` // 20

4.4 Compound Assignment Operators

Combine an arithmetic operation with assignment.

Operator	Meaning	Example	Equivalent To
+=	Add and assign	<code>a += 5</code>	<code>a = a + 5</code>
-=	Subtract and assign	<code>a -= 5</code>	<code>a = a - 5</code>
*=	Multiply and assign	<code>a *= 5</code>	<code>a = a * 5</code>
/=	Divide and assign	<code>a /= 5</code>	<code>a = a / 5</code>
%=	Modulus and assign	<code>a %= 5</code>	<code>a = a % 5</code>

4.5 Relational Operators

Used to compare two values. Result is a boolean value (true/false).

Operator	Meaning	Example
>	Greater than	<code>a > b</code>
<	Less than	<code>a < b</code>
>=	Greater than or equal to	<code>a >= b</code>
<=	Less than or equal to	<code>a <= b</code>
=	Equal to	<code>a == b</code>
!=	Not equal to	<code>a != b</code>

Example:

```
int a = 5, b = 10;
System.out.println(a > b); // false
System.out.println(a != b); // true
```